CLOUD & PLATFORMS

SAP BTP

Build and extend SAP applications the clean-core way — environments, the CAP model, destinations, XSUAA auth, side-by-side extensibility, and Fiori/UI5.

sap-btp · cap · cloud-foundry · xsuaa · fiori · kyma · destinations

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What SAP BTP Is & Why It Matters
2. Account Model & Getting Started
3. The Three Runtime Environments
4. The Cloud Application Programming (CAP) Model
5. Destinations & Connectivity
6. XSUAA — Authentication & Authorization
7. Extension vs Side-by-Side
8. Fiori / UI5 in the BTP Picture
9. Deploying to BTP: MTA
10. Cheat-Sheet & Comparisons
11. Best Practices & Common Pitfalls
12. Interview Questions & Answers
13. Further Learning

What SAP BTP Is & Why It Matters

SAP Business Technology Platform (BTP) is SAP's unified Platform-as-a-Service for building, extending, and integrating applications around a company's SAP core (S/4HANA, SuccessFactors, Ariba, and more) — without modifying that core.

BTP bundles four technology pillars under one roof: **application development** (runtimes, the CAP model, Fiori/UI5), **integration** (Integration Suite, APIs, events), **data & analytics** (SAP HANA Cloud, Datasphere, SAP Analytics Cloud), and **AI** (SAP AI Core, Joule, Generative AI Hub). The strategic

promise is the *clean core*: keep your ERP standard and upgradable, and push all customization into a governed platform alongside it.

EXTEND, DON'T MODIFY

Add custom logic and UIs *next to* S/4HANA instead of patching it, so upgrades stay painless and your core stays "clean."

ONE ACCOUNT, MANY SERVICES

A single global account gives entitlement-based access to dozens of services across multiple hyperscaler regions (AWS, Azure, GCP, AliCloud).

OPEN & STANDARDS-BASED

Cloud Foundry, Kubernetes (Kyma), OData, OAuth 2.0, OpenAPI, CloudEvents — not a proprietary silo.

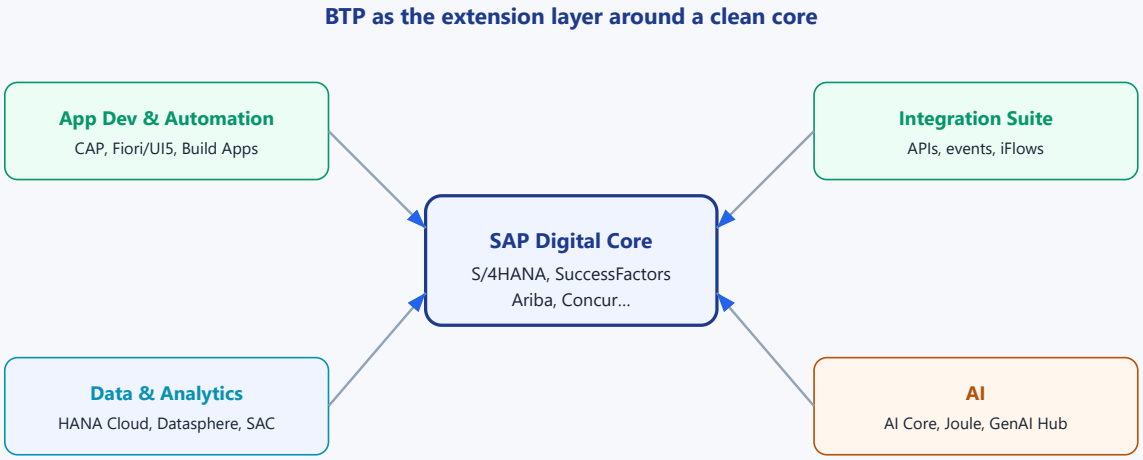


Fig 1 — BTP services surround the digital core; extensions live outside it, connected through destinations and events.

Mental model

Think of BTP as "the App Store, dev environment, and integration bus for your SAP landscape." Your ERP is the system of record; BTP is where you build everything that would otherwise be a risky core modification.

Account Model & Getting Started

Before you deploy anything you must understand the BTP account hierarchy. It is the governance and billing backbone, and interviewers love to probe it.

Global Account -> **Directory (optional)** -> **Subaccount** -> **Space (CF) / Namespace (Kyma)**

GLOBAL ACCOUNT

The top contract with SAP. Holds your entitlements and quotas, and is tied to a commercial model (consumption-based CPEA/BTPEA or subscription).

DIRECTORY

Optional grouping of subaccounts for shared entitlements, custom properties, and authorization inheritance.

SUBACCOUNT

The real working unit: pinned to a *region* and *provider*, carries its own trust config, destinations, and enabled environment (CF/Kyma/ABAP).

SPACE / NAMESPACE

Within a subaccount: a Cloud Foundry *space* or a Kyma *namespace* isolates deployed apps and their service instances.

Install the two CLIs you will use daily — the BTP CLI (account/platform management) and the Cloud Foundry CLI (app lifecycle):

BASH

```
# btp CLI — manage global accounts, subaccounts, entitlements, trust
btp login --url https://cli.btp.cloud.sap
btp list accounts/subaccount
btp assign security/role-collection "Subaccount Administrator" --to-user you@corp.com

# Cloud Foundry CLI — target an org/space and push apps
cf login -a https://api.cf.eu10-004.hana.ondemand.com
cf target -o my-org -s dev
cf marketplace            # list available services (plans)
cf services               # instances already created
```

For local development you install the CAP toolkit and the MTA build tool:

BASH

```bash
# Node.js CAP development kit (the `cds` command)
npm install -g @sap/cds-dk
cds --version

# Multi-Target Application (MTA) build tool — packages apps for BTP
npm install -g mbt
npm install -g @sap/approuter   # the application router
```

✦ **Trial vs free tier**

Use the **BTP free tier** (within a productive global account, cancellable services) or a **trial account** to learn. Free-tier service plans are real services with capacity caps — perfect for a portfolio project.

The Three Runtime Environments

A subaccount can enable one or more *environments*. Choosing between them is a classic design decision.

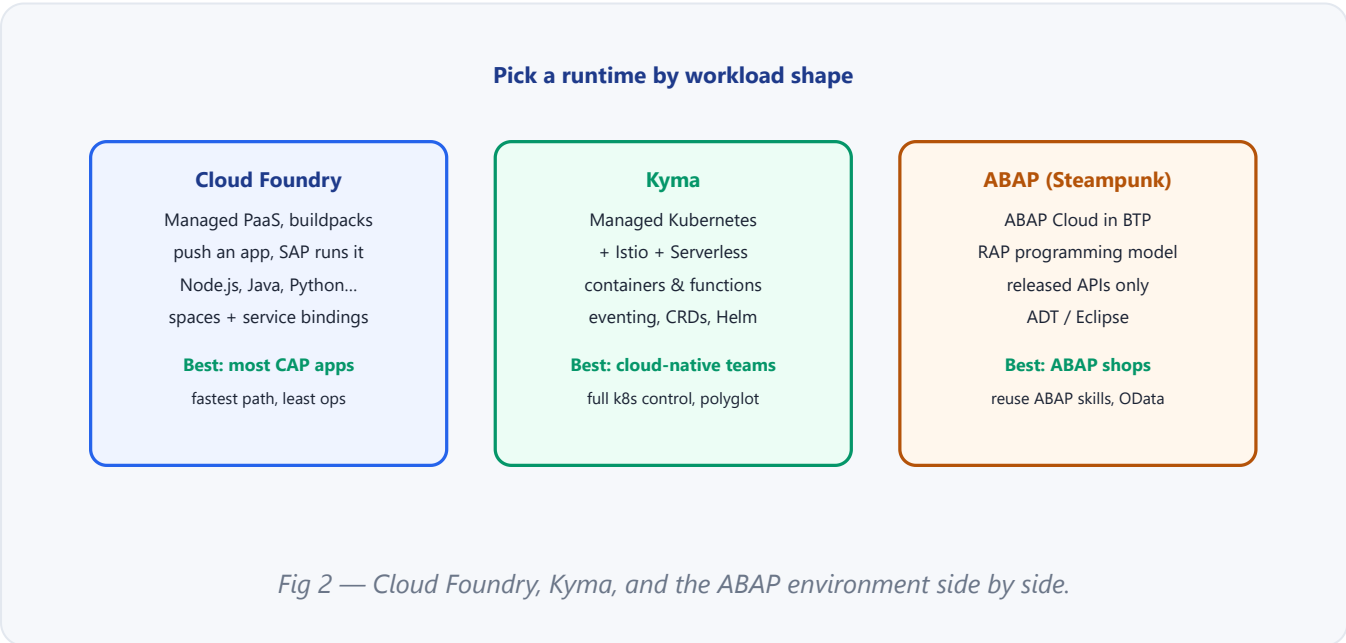


Fig 2 — Cloud Foundry, Kyma, and the ABAP environment side by side.

CLOUD FOUNDRY

The default and most mature environment. You `cf push` an application; buildpacks detect the language and produce a running container. Services are consumed via *service instances* bound to the app, which inject credentials as the `VCAP_SERVICES` environment variable.

KYMA

A fully managed Kubernetes cluster with Istio service mesh, API Gateway, and a serverless Function engine. You deploy standard container images and Helm charts, define `APIRule` resources to expose services, and use SAP BTP Service Operator to provision BTP services as Kubernetes resources.

ABAP ENVIRONMENT

Nicknamed "Steampunk," this is ABAP in the cloud. You develop in ABAP Development Tools (ADT) using the **RAP** (RESTful Application Programming) model and may only call *released* SAP APIs — enforcing the clean-core discipline at the language level. Ideal for teams with deep ABAP investment.

The Cloud Application Programming (CAP) Model

CAP is SAP's opinionated, framework for building services and apps. You model data and services declaratively in **CDS** (Core Data Services), and CAP generates a running OData/REST service with sensible defaults — you add custom logic only where needed.

CAP comes in two flavors: **Node.js** (`@sap/cds`) and **Java** (CAP Java SDK). The CDS modeling language is identical across both. A CAP project has a conventional layout:

BASH

```
my-cap-app/
├── db/            # domain model (entities) + .csv seed data
│   └── schema.cds
├── srv/           # service definitions + custom handlers
│   ├── cat-service.cds
│   └── cat-service.js
├── app/           # UI modules (Fiori/UI5), Fiori preview config
├── package.json   # cds config, dependencies
└── mta.yaml       # deployment descriptor (generated by `cds add mta`)
```

1. DEFINE THE DOMAIN MODEL (CDS)

CDS

```
// db/schema.cds
namespace my.bookshop;

using { managed, cuid } from '@sap/cds/common';

entity Books : cuid, managed {
  title  : String(111);
  stock  : Integer;
  price  : Decimal(9,2);
  author : Association to Authors;   // managed foreign-key association
}

entity Authors : cuid, managed {
  name  : String(111);
  books : Association to many Books on books.author = $self;
}
```

2. EXPOSE A SERVICE

CDS

```
// srv/cat-service.cds
using my.bookshop as my from '../db/schema';

service CatalogService @(path: '/browse') {
  @readonly entity Books   as projection on my.Books;
            entity Authors as projection on my.Authors;

  // an unbound action - custom business operation
  action submitOrder(book : UUID, quantity : Integer) returns { stock : Integer };
}
```

3. ADD CUSTOM LOGIC WITH EVENT HANDLERS

CAP's runtime is event-driven: every CRUD request and action fires events you can hook with `before`, `on`, and `after` handlers.

JAVASCRIPT

```
// srv/cat-service.js
const cds = require('@sap/cds')

module.exports = class CatalogService extends cds.ApplicationService {
  init() {
    const { Books } = this.entities

    // Validate before every CREATE/UPDATE on Books
    this.before(['CREATE', 'UPDATE'], Books, (req) => {
      if (req.data.stock < 0) req.error(400, 'Stock must not be negative')
    })

    // Enrich each row after reading
    this.after('READ', Books, (books) => {
      for (const b of books) if (b.stock < 5) b.title += ' (low stock!)'
    })

    // Custom action implementation
    this.on('submitOrder', async (req) => {
      const { book, quantity } = req.data
      const row = await SELECT.one.from(Books).where({ ID: book })
      if (!row) return req.error(404, 'Book not found')
      if (quantity > row.stock) return req.error(409, 'Not enough stock')
      await UPDATE(Books).set({ stock: { '-=': quantity } }).where({ ID: book })
      return { stock: row.stock - quantity }
    })

    return super.init()
  }
}
```

4. RUN IT LOCALLY

BASH

```
cds watch        # starts an in-memory SQLite DB, serves OData, hot-reloads
# Browse:  http://localhost:4004/browse/Books
# OData:   http://localhost:4004/browse/$metadata
```

✦ **Why CAP wins**

You wrote ~20 lines of model and got a full OData v4 service with CRUD, deep-insert, pagination (`$top/$skip`), filtering (`$filter`), draft handling, and input validation — plus a path to HANA with zero rewrite.

The CAP request lifecycle is worth internalizing:

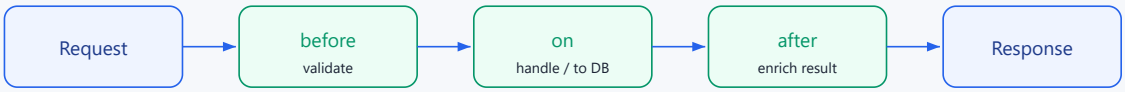


Fig 3 — CAP event phases: before (validation), on (persistence/custom), after (post-processing).

Destinations & Connectivity

A **destination** is a reusable, centrally managed definition of a remote endpoint — URL, authentication type, and credentials. Your app never hard-codes a system's URL or secrets; it asks the Destination service for a destination by name at runtime.

JSON

```
// A destination configured in the BTP cockpit (shown as its JSON form)
{
  "Name": "S4HANA_API",
  "URL": "https://my-s4.s4hana.ondemand.com",
  "Type": "HTTP",
  "ProxyType": "Internet",
  "Authentication": "OAuth2SAMLBearerAssertion",  // principal propagation
  "tokenServiceURL": "https://.../oauth/token",
  "clientId": "sb-...",
  "sap-client": "100"
}
```

CAP consumes destinations transparently when you declare a remote service and bind the destination + connectivity services:

JAVASCRIPT

```
// package.json → cds.requires
// "S4bupa": { "kind": "odata-v2", "credentials": { "destination": "S4HANA_API" } }

const s4 = await cds.connect.to('S4bupa')
const partners = await s4.run(
  SELECT.from('A_BusinessPartner').limit(10)
)   // CAP builds the HTTP call, resolves the destination, attaches the token
```

> **On-premise reach**
>
> To call systems behind a corporate firewall, pair the Destination service with the **Cloud Connector** (a lightweight on-prem agent) and set `ProxyType: OnPremise`. The Connectivity service tunnels the request securely — no inbound firewall holes.

XSUAA — Authentication & Authorization

XSUAA (Extended Services for UAA) is BTP's OAuth 2.0 authorization server. It issues JWT access tokens, maps your Identity Provider's users to application-defined *scopes* and *role collections*, and is the backbone of security on Cloud Foundry.

You declare your app's security model in `xs-security.json`: scopes (atomic permissions), attributes, and role templates.

JSON

```
// xs-security.json
{
  "xsappname": "bookshop",
  "tenant-mode": "dedicated",
  "scopes": [
    { "name": "$XSAPPNAME.Read",  "description": "Read books" },
    { "name": "$XSAPPNAME.Admin", "description": "Manage catalog" }
  ],
  "role-templates": [
    { "name": "Viewer", "scope-references": ["$XSAPPNAME.Read"] },
    { "name": "Administrator",
      "scope-references": ["$XSAPPNAME.Read", "$XSAPPNAME.Admin"] }
  ]
}
```

In CDS you annotate services with `@requires` and `@restrict`; CAP enforces the scopes from the JWT automatically:

CDS

```
service CatalogService {
  @requires: 'authenticated-user'
  entity Books @(restrict: [
    { grant: 'READ',  to: 'Viewer' },
    { grant: ['CREATE','UPDATE','DELETE'], to: 'Administrator' }
  ]) as projection on my.Books;
}
```

The runtime flow ties the Approuter, XSUAA, and your service together:

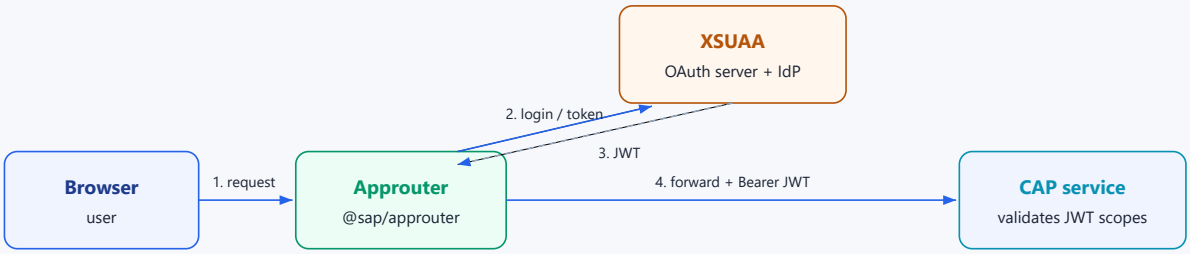


Fig 4 — The Approuter authenticates users at XSUAA, then forwards requests with a JWT the backend validates against required scopes.

> ⚠ **XSUAA vs IAS**
> XSUAA handles *authorization* (scopes/roles) for apps. SAP **Identity Authentication Service (IAS)** is the *authentication* IdP for users across the whole tenant. Modern BTP setups increasingly front XSUAA with IAS; know the distinction.

Extension vs Side-by-Side

"Extensibility" is BTP's reason to exist. There are three recognized tiers — interviewers expect you to name and contrast them.

KEY-USER (IN-APP)

No-code/low-code adaptations *inside* S/4HANA by a business expert: custom fields, UI layout tweaks, custom logic via released BAdIs. Stays in the core but upgrade-safe.

DEVELOPER (IN-STACK)

ABAP Cloud developments *on* the S/4HANA stack (or the BTP ABAP environment), restricted to released APIs. More power, still governed.

SIDE-BY-SIDE

A separate app *on BTP* (CAP/Fiori/Kyma) that integrates via OData APIs, events, and destinations. Maximum freedom, independent lifecycle, polyglot.



Fig 5 — Side-by-side keeps the core untouched; integration happens over APIs and events.

✓ **Rule of thumb**

If it changes core data behavior and is small, extend in-app/in-stack. If it has its own UI, data, scaling needs, or non-ABAP tech, build it side-by-side on BTP.

Fiori / UI5 in the BTP Picture

SAP Fiori is the design language; SAPUI5 is the JavaScript framework that implements it. On BTP, a CAP OData service plus a Fiori **elements** app is the canonical full-stack combo — the UI is largely generated from OData annotations rather than hand-coded.

JAVASCRIPT

```
// Scaffold a Fiori elements app over your CAP service
// (via the SAP Fiori generator in VS Code / Business Application Studio)
// Annotations drive the List Report + Object Page UI:
```

CDS

```cds
// app/annotations.cds — no imperative UI code needed
using CatalogService as service from '../srv/cat-service';

annotate service.Books with @(
  UI: {
    LineItem: [
      { Value: title },
      { Value: stock },
      { Value: price }
    ],
    HeaderInfo: { TypeName: 'Book', TypeNamePlural: 'Books', Title: { Value: title } }
  }
);
```

Freestyle UI5 (hand-written XML views + controllers) is used when you need full control:

XML

```xml
<mvc:View controllerName="bookshop.controller.List"
  xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m">
  <Table items="{/Books}">
    <columns><Column><Text text="Title"/></Column></columns>
    <items><ColumnListItem><cells>
      <Text text="{title}"/>
    </cells></ColumnListItem></items>
  </Table>
</mvc:View>
```

Finished apps are published to the **SAP Build Work Zone** (formerly Launchpad service), which gives users a single tile-based entry point across all their Fiori apps.

Deploying to BTP: MTA

Multi-Target Applications bundle every module — the CAP service, the Approuter, the UI, the HANA DB deployer — into one deployable archive described by `mta.yaml`.

YAML

```yaml
# mta.yaml (abridged)
ID: bookshop
version: 1.0.0
modules:
  - name: bookshop-srv            # CAP service
    type: nodejs
    path: gen/srv
    requires:
      - name: bookshop-db
      - name: bookshop-auth
  - name: bookshop-db-deployer    # pushes schema to HANA Cloud
    type: hdb
    path: gen/db
    requires: [{ name: bookshop-db }]
  - name: bookshop-approuter
    type: approuter.nodejs
    path: app/router
    requires: [{ name: bookshop-auth }]
resources:
  - name: bookshop-db
    type: com.sap.xs.hdi-container     # HANA HDI container
  - name: bookshop-auth
    type: org.cloudfoundry.managed-service
    parameters: { service: xsuaa, service-plan: application }
```

BASH

```bash
cds add hana,xsuaa,approuter,mta   # wire up the deployment artifacts
mbt build                          # produces mta_archives/bookshop_1.0.0.mtar
cf deploy mta_archives/bookshop_1.0.0.mtar   # needs the MultiApps CF plugin
```

Cheat-Sheet & Comparisons

Command	What it does
`btp login --url ...`	Authenticate the BTP CLI to a region
`btp list accounts/subaccount`	List subaccounts in the global account
`cf push`	Build & deploy an app to Cloud Foundry
`cf create-service xsuaa application my-auth`	Provision an XSUAA instance
`cf bind-service my-app my-auth`	Inject service credentials into an app
`cds watch`	Run the CAP app locally with hot reload
`cds add hana,xsuaa,mta`	Add production deployment artifacts
`mbt build`	Build the MTA archive (.mtar)
`cf deploy app.mtar`	Deploy a multi-target application

Aspect	Cloud Foundry	Kyma	ABAP Env.
Abstraction	PaaS / buildpacks	Managed Kubernetes	ABAP platform
Deploy unit	`cf push` / MTA	Container + Helm	Transport / ADT
Languages	Node, Java, Python…	Any (containers)	ABAP Cloud
Ops burden	Lowest	Higher (k8s skills)	Low (SAP-managed)
Best fit	Most CAP apps	Cloud-native, eventing	ABAP teams, RAP

Concept	One-line definition
Entitlement	The *right* to use a service plan, assigned to a subaccount
Service instance	A provisioned, bindable instance of a service
Destination	Named, managed definition of a remote endpoint + auth
Approuter	Single entry point handling auth, routing, static content
Role collection	Bundle of roles assigned to users/groups in a subaccount
HDI container	Isolated schema in SAP HANA Cloud for one app

Best Practices & Common Pitfalls

✓ **Keep the core clean**

Default to side-by-side or released-API extensions. Never modify standard SAP objects; it breaks upgrades and support.

✦ **Externalize everything**

Use destinations for URLs/credentials and user-provided or service bindings for secrets. No hard-coded endpoints or passwords in code.

✦ **Multitenancy early**

If you may sell your extension to multiple customers, design for SaaS multitenancy from day one — retrofitting `tenant-mode: shared` is painful.

⚠ **Mind the quotas**

Entitlements are finite. A failed deploy is often just an unassigned entitlement or an exhausted quota in the subaccount, not a code bug.

⚠ **Don't skip the Approuter**

Calling a CAP service directly in production bypasses central auth and CSRF handling. The Approuter (or Work Zone) must front user-facing apps.

Token & trust pitfalls

JWT audience/scope mismatches cause silent 403s. Verify `xsappname`, that the right role collections are assigned, and that your subaccount trusts the correct IdP.

Interview Questions & Answers

What problem does SAP BTP solve?

It provides a governed platform to extend and integrate SAP systems *without* modifying the core, enabling the "clean core" strategy: the ERP stays standard and upgradable while customization lives on BTP via app dev, integration, data/analytics, and AI services.

Explain the BTP account hierarchy.

Global account (commercial contract + entitlements) → optional directories (grouping) → subaccounts (region/provider-bound working units with their own trust, destinations, environments) → spaces (Cloud Foundry) or namespaces (Kyma).

Compare the three runtime environments.

Cloud Foundry: managed PaaS via buildpacks, lowest ops, best for most CAP apps. Kyma: managed Kubernetes + Istio + serverless, for cloud-native polyglot workloads. ABAP (Steampunk): ABAP Cloud with the RAP model and released APIs only, for ABAP-centric teams.

What is the CAP model and why use it?

Cloud Application Programming is SAP's framework where you model data/services declaratively in CDS and get a full OData/REST service with CRUD, pagination, filtering, draft handling, and validation out of the box. You add imperative logic only via before/on/after event handlers. It runs on Node.js or Java and deploys cleanly to HANA.

What is a destination and why not just hard-code URLs?

A destination is a centrally managed definition of a remote endpoint (URL, auth type, credentials). It decouples code from environment config, lets admins rotate secrets without redeploying, and — with the Cloud Connector — securely reaches on-premise systems via `ProxyType: OnPremise`.

What is XSUAA and how does auth flow work?

XSUAA is BTP's OAuth 2.0 authorization server. The Approuter authenticates the user (often via IAS), obtains a JWT, and forwards requests with it. The backend validates the JWT and checks scopes defined in `xs-security.json`. Scopes map to role templates → roles → role collections assigned to users.

> **Difference between XSUAA and IAS?**
>
> IAS (Identity Authentication Service) authenticates *who the user is* (the IdP). XSUAA handles *what they can do* (authorization scopes/roles) for applications. Modern setups front XSUAA with IAS for unified authentication.

> **Explain extension vs side-by-side.**
>
> Key-user and in-stack (ABAP Cloud) extensions live on the core, limited to released APIs, upgrade-safe but constrained. Side-by-side runs a separate app on BTP integrating via OData/events/destinations — maximum freedom, independent lifecycle and scaling, polyglot tech.

> **What does the Approuter do?**
>
> It is the single entry point for a BTP app: handles user authentication against XSUAA, manages login sessions and CSRF tokens, serves static UI content, and reverse-proxies routes to backend microservices with the JWT attached.

> **How do you deploy a full CAP+UI app to BTP?**
>
> Run `cds add hana,xsuaa,approuter,mta` to generate deployment artifacts, `mbt build` to produce an `.mtar` archive bundling the service, DB deployer, Approuter, and UI, then `cf deploy` it. The MTA descriptor wires modules to resources like the HDI container and XSUAA instance.

> **What is Fiori elements vs freestyle UI5?**
>
> Fiori elements generates standardized floorplans (List Report, Object Page) from OData annotations — little custom JS, consistent UX, fast. Freestyle UI5 is hand-coded XML views and controllers for full control when a scenario doesn't fit the standard templates.

> **How does CAP connect to a remote S/4HANA service?**
>
> Declare a required remote service in `cds.requires` pointing at a destination, then `cds.connect.to('Service')` and run CQL queries against it. CAP builds the HTTP/OData call, resolves the destination, and attaches the token (e.g. principal propagation via OAuth2SAMLBearerAssertion).

Further Learning

- Work through the official **CAP (capire)** documentation and build the bookshop sample end to end on the free tier.
- Complete the SAP **Learning Journey** "Developing Applications on SAP BTP" and the CAP + Fiori elements tutorials on the SAP Tutorial Navigator.

- Deploy a side-by-side extension that reads S/4HANA Business Partners via a destination and the Cloud Connector.
- Explore Kyma: containerize a service, expose it with an `APIRule`, and trigger a serverless Function from a business event.
- Study **RAP** (RESTful ABAP Programming) if you work in ABAP shops, and the **SAP Integration Suite** for iFlows and API management.
- Target the **SAP Certified Development Associate — SAP BTP Extensions** (CAP, Fiori) certification to validate your skills.